МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Редакционное расстояние

Студентка гр. 4345 _____ Жилина А. Е.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Задание

Над строкой εε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. replace(εε, a, b) – заменить символ a на символ b.
2. insert(εε, a) – вставить в строку символ a (на любую позицию).
3. delete(εε, b) – удалить из строки символ b.

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки AA и BB, а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки AA в строку BB.

Входные данные: первая строка – три числа: цена операции replace, цена операции insert, цена операции delete; вторая строка – AA; третья строка – BB.

Выходные данные: одно число – минимальная стоимость операций.

Sample Input:

1 1 1

entrance

reenterable

Sample Output:

5

Индивидуальный вариант: 2. Особый заменитель и особо удаляемый символ": цена замены на определённый символ отличается от обычной цены замены; цена удаления другого (или того же) определённого символа отличается от обычной цены удаления. Особый заменитель и цена замены на него, особо удаляемый символ и цена его удаления — дополнительные входные данные.

Описание алгоритма

Основная идея алгоритма заключается в использовании восходящего динамического программирования (bottom-up DP) для вычисления минимальной стоимости трансформации одной строки в другую.

В отличие от жадных подходов, которые могут совершать локально оптимальные, но глобально ошибочные выборы, алгоритм не пытается угадать правильную последовательность действий сразу. Вместо этого задача разбивается на сетку пересекающихся подзадач. Алгоритм последовательно заполняет двумерную матрицу, где каждая ячейка dp[i][j] хранит минимальную стоимость преобразования префикса первой строки (длиной i) в префикс второй строки (длиной j). Значение в каждой новой клетке вычисляется строго на основе трех уже известных предыдущих состояний, соответствующих операциям удаления, вставки и замены. Благодаря этому гарантируется, что итоговый ответ в правой нижней клетке матрицы является абсолютно оптимальным.

Сложность по времени: $O(N \cdot M)$

Основной цикл: Программа использует два вложенных цикла, проходящих по длинам исходной строки N (строка A) и целевой строки M (строка B). Это дает ровно $N \cdot M$ итераций основной логики.

Вспомогательные операции: На каждом шаге алгоритм обращается к функциям get_del_cost, get_ins_cost и get_rep_cost, которые отрабатывают за константное время $O(1)$. Проверка условий и нахождение минимума из трех вариантов также занимают $O(1)$. Логирование в файл добавляет константное время на каждую ячейку.

Восстановление пути (бэктрекинг): Если требуется получить саму последовательность операций, обратный проход от конца матрицы к её началу занимает $O(N + M)$ шагов в худшем случае.

Общая теоретическая сложность $O(N \cdot M) + O(N + M) = O(N \cdot M)$

Сложность по памяти: $O(N \cdot M)$

Матрица состояний: Основной объем памяти выделяется под двумерный массив dp размером $(N+1) \times (M+1)$. Алгоритм сохраняет всю матрицу целиком, так как её полная история необходима для последующего восстановления пути (вывода строкового лога операций).

Стек и переменные: Глубина стека вызовов минимальна (алгоритм итеративный, рекурсия не используется). Массив операций ops для хранения ответа занимает не более $O(N+M)$ памяти.

Итоговая сложность: $O(N \cdot M)$. Если бы восстановление пути не требовалось, алгоритм можно было бы оптимизировать до $\mathrm{O}(\min(\mathrm{N}, \mathrm{M}))$, храня только две последние строки матрицы, но текущая реализация жертвует памятью ради полноты данных.

Особенности и модификации алгоритма

Несмотря на использование классической матрицы Вагнера — Фишера, данная реализация включает ряд специфических особенностей под расширенные требования:

1. Динамическое ценообразование операций:
   В отличие от базового расстояния Левенштейна, где любая операция стоит ровно 1, алгоритм включает систему гибких весов. Использование функций get_del_cost и get_rep_cost позволяет задавать индивидуальные цены не только для типов операций (вставка/удаление), но и для конкретных символов (переменные spec_rep_char и spec_del_char). Это позволяет легко адаптировать алгоритм к сценариям, где, например, опечатка в конкретной букве обходится «дешевле» или «дороже» остальных.
2. Детерминированный бэктрекинг:
   При восстановлении пути по матрице программа строго приоритизирует диагональные переходы (совпадения и замены M или R) перед вертикальными и горизонтальными (удаления D и вставки I). Это позволяет получать наиболее логичную последовательность правок без лишних сдвигов.

3. Интегрированная система аудита (логирование):
   В алгоритм встроен механизм параллельной записи состояния матрицы на каждом шаге инициализации и вычисления. Это позволяет визуально отслеживать, как именно меняются стоимости в ячейках при применении кастомных штрафов на конкретные символы, что критически важно для проверки правильности эвристик.

Способ хранения промежуточных состояний

Для хранения промежуточных состояний (уже вычисленных стоимостей преобразования) алгоритм не создает дерево рекурсивных вызовов, так как классический перебор строк привел бы к экспоненциальному дублированию вычислений. Вместо этого используется единый двумерный массив (матрица) dp размером $(n + 1) \times (m + 1)$. В этом массиве каждая ячейка dp[i][j] сохраняет уже найденное минимальное количество штрафных баллов для превращения подстроки длины i в подстроку длины j.

Вектор (список) ops выступает в роли хранилища маршрута при восстановлении ответа. Он работает по принципу стека: алгоритм, двигаясь от конца матрицы к её началу, добавляет в конец списка идентификаторы операций (где 'M' — совпадение, 'R' — замена, 'D' — удаление, 'I' — вставка). Поскольку маршрут восстанавливается с конца, по завершении обратного прохода список ops просто разворачивается (операция reverse()), выдавая хронологически верную последовательность действий для транскрипта.

Описание итеративной функции и обратного хода (бэктрекинг)

Основа алгоритма — итеративная функция get_levenshtein_distance. В отличие от рекурсивных подходов, она не пытается «угадать» путь сверху вниз, а последовательно выстраивает фундамент решений снизу вверх с помощью двух вложенных циклов.

После инициализации нулевых строки и столбца (базовые случаи, где слово превращается в пустую строку и наоборот), алгоритм сканирует матрицу строго слева направо и сверху вниз. В каждой ячейке программа параллельно рассчитывает стоимости трех потенциальных действий: удаления (del_cost_total), вставки (ins_cost_total) и замены (rep_cost_total), опираясь на кастомные функции цен (get_del_cost, get_ins_cost, get_rep_cost). Функция записывает в dp[i][j] строго минимальное из этих трех значений, навсегда фиксируя оптимальный локальный ответ.

Процесс восстановления пути (бэктрекинг):

Если функция вызвана с флагом return_transcript=True, после заполнения таблицы запускается цикл while i > 0 or j > 0. Его задача — отследить, как именно алгоритм попал в финальную нижнюю правую клетку dp[n][m].

На каждом шаге программа заново вычисляет цены переходов из трех соседних ячеек (по диагонали, сверху и слева) и сравнивает их с текущим значением dp[i][j]. Как только находится точное математическое совпадение (например, если текущая ячейка равна ячейке сверху плюс цена удаления), алгоритм понимает, что шаг был сделан именно оттуда. Индексы i и j соответственно уменьшаются (шаг назад), пока программа не дойдет до стартовой нулевой ячейки, завершая процесс расшифровки.

Структуры данных и ключевые переменные:

A и B (строки) — базовые структуры данных, представляющие исходную (ту, которую исправляем) и целевую (ту, которую хотим получить) последовательности символов. Их длины обозначаются как n и m соответственно.

dp (двумерный список) — основная структура данных алгоритма. Это двумерный массив размером $(n + 1) \times (m + 1)$, представляющий матрицу состояний динамического программирования. В отличие от рекурсивного

дерева, матрица позволяет переиспользовать уже вычисленные минимальные стоимости для префиксов строк, обеспечивая доступ к ним за время $O(1)$.

ops (список) — динамический массив, выполняющий роль стека для текущего пути при восстановлении ответа. Он хронологически накапливает символьные идентификаторы операций (M, R, D, I), двигаясь с конца в начало, чтобы затем быть развернутым в правильную последовательность.

cost_ins, cost_del, cost_rep (целые числа) — набор переменных, хранящих базовые стоимости (штрафы) за выполнение операций вставки, удаления и замены.

Специфичные модификаторы (spec_rep_char, spec_rep_cost, spec_del_char, spec_del_cost) — переменные, задающие индивидуальные правила ценообразования. Позволяют точечно изменять стоимость удаления или замены для конкретных символов-исключений.

Функции алгоритма:

get_del_cost(char), get_rep_cost(char_from, char_to), get_ins_cost(char) — вложенные вспомогательные функции (замыкания) для инкапсуляции логики расчета штрафов.

get_rep_cost сперва проверяет символы на совпадение: если char_from == char_to, функция мгновенно возвращает 0 (операция не требуется). В противном случае проверяется совпадение с особым символом spec_rep_char.

get_del_cost аналогично проверяет удаляемый символ на совпадение со spec_del_char, возвращая либо кастомную, либо базовую стоимость.

Использование этих функций избавляет основной цикл от громоздких условных конструкций (if-else).

log(text), log_matrix() — вспомогательные функции аудита. Отвечают за форматирование и запись текущего состояния переменных и матрицы dp в текстовый файл log.txt. log_matrix динамически генерирует

заголовки строк и столбцов на основе исходных слов, обеспечивая наглядную визуализацию процесса заполнения таблицы.

get_levenshtein_distance(A, B, ...) — основная вычислительная функция, реализующая алгоритм Вагнера — Фишера. Возвращает минимальную стоимость преобразования (и, опционально, строку-транскрипт).

Аргументы:

Принимает исходную и целевую строки, все базовые и специальные стоимости операций, а также флаг return_transcript, указывающий, требуется ли выполнять бэктрекинг.

Базовые случаи (Инициализация):

С помощью двух независимых циклов функция заполняет нулевой столбец и нулевую строку матрицы dp. Это соответствует пограничным случаям: превращению строки в пустую (только удаления) и созданию строки из пустой (только вставки). Каждая ячейка опирается на значение предыдущей плюс соответствующую стоимость операции.

Заполнение основной таблицы:

С помощью двойного цикла алгоритм сканирует оставшуюся матрицу. Для каждой клетки dp[i][j] вычисляются три суммарные стоимости: del_cost_total (значение ячейки сверху + цена удаления), ins_cost_total (значение ячейки слева + цена вставки) и rep_cost_total (значение ячейки по диагонали + цена замены/совпадения). В текущую клетку записывается строгий минимум из этих трех вариантов.

Восстановление пути (Бэктрекинг):

Если флаг return_transcript установлен в True, запускается цикл while. Алгоритм начинает движение из ячейки dp[n][m] обратно к dp[0][0]. На каждом шаге он проверяет, из какой соседней ячейки был совершен оптимальный переход, добавляет соответствующий символ операции в массив ops и сдвигает индексы i и j.

Главный блок программы (if __name__ == "__main__") — управляющая логика, заменяющая функцию main() в языках со статической типизацией.

Считывает через стандартный ввод (консоль) строки A и B, базовые цены операций и параметры для особых символов.

Вызывает функцию get_levenshtein_distance, передавая в неё собранные параметры в виде именованных аргументов.

Выводит итоговое минимальное количество исправлений в консоль и уведомляет пользователя об успешном сохранении подробного лога вычислений в файл.

Тестирование

Результаты тестирования представлены в табл. 1.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные	Комментарии
1.	Строка 1: ВАСИЛЬЕВ Строка 2: КВАСЮТИН Цена вставки: 1 Цена удаления: 1 Цена замены: 1 Особый заменитель: - Цена замены на символ: 0 Особо удаляемый символ: - Цена удаления символа: 0	6	Верный результат
2.	Строка 1: ИВАНОВ Строка 2: ИВАНОВ Цена вставки: 1 Цена удаления: 1 Цена замены: 1 Особый заменитель: - Цена замены на символ: 0 Особо удаляемый символ: - Цена удаления символа: 0	0	Верный результат

3.	Строка 1: Строка 2: АБВ Цена вставки: 2 Цена удаления: 1 Цена замены: 1 Особый заменитель: - Цена замены на символ: 0 Особо удаляемый символ: - Цена удаления символа: 0	6	Верный результат
4.	Строка 1: (пустая строка) Строка 2: АБВ Цена вставки: 2 Цена удаления: 1 Цена замены: 1 Особый заменитель: - Цена замены на символ: 0 Особо удаляемый символ: - Цена удаления символа: 0	12	Верный результат

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.py

```
def get_levenshtein_distance(A, B, cost_rep=1, cost_ins=1, cost_del=1,
                             spec_rep_char=None, spec_rep_cost=None,
                             spec_del_char=None, spec_del_cost=None,
                               return_transcript=False):
    def get_del_cost(char):
        if spec_del_char is not None and char == spec_del_char:
            return spec_del_cost
        return cost_del

    def get_rep_cost(char_from, char_to):
        if char_from == char_to:
            return 0
        if spec_rep_char is not None and char_to == spec_rep_char:
            return spec_rep_cost
        return cost_rep

    def get_ins_cost(char):
        return cost_ins

    n, m = len(A), len(B)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    with open("log.txt", "w", encoding="utf-8") as log_file:
        def log(text):
            log_file.write(text + "\n")

        def log_matrix():
            header = "      " + " ".join(f"{ch:>3}" for ch in B)
            log(header)

            for r in range(n + 1):
                row_char = " " if r == 0 else A[r - 1]
                  row_vals = " ".join(f"{dp[r][c]:>3}" for c in range(m + 1))
```

```
                    log(f"{row_char} {row_vals}")
                log("")

            log(f"Начало вычисления расстояния Левенштейна.")
            log(f"Исходная строка: '{A}' (длина {n})")
            log(f"Целевая строка: '{B}' (длина {m})\n")

            # Инициализация базовых случаев
            log("--- Инициализация нулевого столбца ---")
            for i in range(1, n + 1):
                cost = get_del_cost(A[i - 1])
                dp[i][0] = dp[i - 1][0] + cost
                log(f"dp[{i}][0] = {dp[i][0]} (Удаление '{A[i - 1]}',
стоимость: {cost})")

            log("\nМатрица после инициализации нулевого столбца:")
            log_matrix()

            log("--- Инициализация нулевой строки ---")
            for j in range(1, m + 1):
                cost = get_ins_cost(B[j - 1])
                dp[0][j] = dp[0][j - 1] + cost
                log(f"dp[0][{j}] = {dp[0][j]} (Вставка '{B[j - 1]}',
стоимость: {cost})")

            log("\nМатрица после инициализации нулевой строки:")
            log_matrix()

            log("--- Заполнение основной таблицы ДП ---")
            # Заполнение таблицы ДП
            for i in range(1, n + 1):
                for j in range(1, m + 1):
                    del_cost_total = dp[i - 1][j] + get_del_cost(A[i -
1])
                    ins_cost_total = dp[i][j - 1] + get_ins_cost(B[j -
1])  # Вставка
                   rep_cost_total = dp[i - 1][j - 1] + get_rep_cost(A[i
- 1], B[j - 1])  # Замена (или совпадение)
```

```
                        dp[i][j] = min(del_cost_total, ins_cost_total,
rep_cost_total)

                    log(f"---")
                    log(f"Клетка [{i}][{j}] | Сравниваем '{A[i - 1]}'
и '{B[j - 1]}':")
                    log(f"  1. Удаление '{A[i - 1]}': {dp[i - 1][j]} +
{get_del_cost(A[i - 1])} = {del_cost_total}")
                    log(f"  2. Вставка '{B[j - 1]}': {dp[i][j - 1]} +
{get_ins_cost(B[j - 1])} = {ins_cost_total}")
                    log(f"  3. Замена/Совпадение: {dp[i - 1][j - 1]} +
{get_rep_cost(A[i - 1], B[j - 1])} = {rep_cost_total}")
                    log(f"  Минимум для dp[{i}][{j}]: {dp[i][j]}")
                    log(f"Текущая матрица:")
                    log_matrix()

            log(f"Итоговое расстояние Левенштейна: {dp[n][m]}")

        if not return_transcript:
            return dp[n][m]

        # Восстановление пути
        ops = []
        i, j = n, m
        while i > 0 or j > 0:
             if i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] +
get_rep_cost(A[i - 1], B[j - 1]):
                ops.append('M' if A[i - 1] == B[j - 1] else 'R')
                i -= 1
                j -= 1
            elif i > 0 and dp[i][j] == dp[i - 1][j] + get_del_cost(A[i
- 1]):
                ops.append('D')
                i -= 1
            elif j > 0 and dp[i][j] == dp[i][j - 1] + get_ins_cost(B[j
- 1]):
                ops.append('I')
                j -= 1
```

```
        ops.reverse()
        return dp[n][m], "".join(ops)

    if __name__ == "__main__":
        A = input("Строка 1: ").strip()
        B = input("Строка 2: ").strip()

        cost_ins = int(input("Цена вставки: "))
        cost_del = int(input("Цена удаления: "))
        cost_rep = int(input("Цена замены: "))

        spec_rep_char = input("Особый заменитель: ").strip()
            spec_rep_cost = int(input(f"Цена замены на символ
'{spec_rep_char}': "))

        spec_del_char = input("Особо удаляемый символ: ").strip()
            spec_del_cost = int(input(f"Цена удаления символа
'{spec_del_char}': "))

        ans = get_levenshtein_distance(
            A, B,
            cost_rep=cost_rep,
            cost_ins=cost_ins,
            cost_del=cost_del,
            spec_rep_char=spec_rep_char,
            spec_rep_cost=spec_rep_cost,
            spec_del_char=spec_del_char,
            spec_del_cost=spec_del_cost
        )

        print(f"Ответ: {ans}")
        print("Подробный процесс вычислений сохранен в файл 'log.txt'.")
```